

Assignment – 6

Date: 15/9/2026

General Instruction

For each question:

1. Implement the required OOP concept.
2. Perform the required tests/output.
3. Perform the specified file-handling operation.
4. Use appropriate file modes and methods/functions covered in the lecture.
5. Properly close files with the appropriate option.
6. Display the output on the screen as well as store the required information in the file.

1. Design Backed support for the HR Access-Control Problem

A company's HR system models employees using a class hierarchy rooted at Employee, with Manager as a specialization. The system has three data-protection requirements:

- The **employee's name** must be freely readable and writable from anywhere in the codebase.
- The **salary** must never be overwritten directly by assigning to it from outside the class. It may only change through a method that revises it by a **percentage hike** between 0 and 50 inclusive; any other value must raise an Error.
- The **leave_balance** must be usable by Employee itself and by any class that extends it, such as Manager, but code outside this class family should not be able to rely on accessing it directly.
- Implement Employee and a Manager subclass with apply_leave(days), which reduces leave_balance and raises an Error if days exceed the remaining balance.

Tasks:

- Implement the Employee class using appropriate access-control conventions.
- Implement the Manager subclass.
- Implement salary revision using a percentage hike between 0 and 50 inclusive.
- Implement apply_leave(days).
- Demonstrate that attaching '__salary' to an Employee instance from outside the class does not read the real salary. Explain what Python actually did to the attribute name.
- Create an employee/manager object and perform some salary and leave operations.
- **File Handling:** Create a text file named employee_records.txt.
- Store the following information in the file:
 - Employee name
 - Salary
 - Remaining leave balance
- Store the information in the file.
- Read the stored information back and display it.
- Properly close the file.

[Encapsulation+ Text File Handling open(), w, write(), read(), close()]

2. Code in Python for an e-commerce **checkout system** needs a common contract for **payment gateways (Credit Card, UPI, and others added later)**. The design must make it structurally impossible to create a generic, gateway-less payment object — every concrete gateway must be forced to supply its own payment logic, while still inheriting a shared receipt-formatting method from the common ancestor.

Tasks:

- Design the **base class** and **two concrete gateways**, [CreditCardProcessor and UPIProcessor] that satisfy the requirement above.
- Show that attempting to instantiate the base class directly fails, and capture/print the exact exception message.

- Process a payment of 1500 through both concrete gateways and print their receipts.
- **File Handling:** Create a text file named payment_receipts.txt.
- Store the receipts generated by both payment processors in the file.
- Use write() to store each receipt.
- Read the file using readlines() and display all stored receipts.

[Abstraction + Text File Handling - w, write(), readlines(), close()]

3. WAP for A notification service must send the same logical message through Email, SMS, and Push channels, but each channel formats the message differently: **Email** tags it, **SMS** truncates the message to its first 20 characters, and **Push** uppercases it. Calling code should never need to know which concrete channel it is talking to.

Tasks:

- Design a common Notifier ancestor whose send(message) is meant to be replaced by every channel.
- Make it fail loudly if a subclass forgets to supply its own version.
- Implement:
 - EmailNotifier
 - SMSNotifier
 - PushNotifier
- Implement the required formatting rules.
- Write a **broadcast(notifiers, message)** function that sends one message through a heterogeneous list using a single loop.
- Run it using: 'Server maintenance scheduled tonight'
- Display the notification generated by each channel.
- **File Handling:** Create a file named **notifications.txt**.
- Store every generated notification in the file.
- Use **append mode (a)** so that new notification messages are added without deleting previously stored notifications.
- Use write() to append each notification.
- Read the complete file and display all stored notifications.

[Method Overriding + File Handling concepts: a, write(), read(), close()]

4. WAP for the **Calculator class** needs a single method named `add` whose behavior depends on the type of arguments provided. The method should accept two or more arguments:

For **integers**, all arguments should be summed arithmetically.

For **strings**, all arguments should be joined using a single space, with Unnecessary leading and trailing spaces removed.

For **lists**, all lists should be merged into a single sorted list containing only unique elements.

Passing mismatched types, such as an integer and a string, should fail clearly.

Tasks:

- If you simply write `def add(self, a, b):` multiple times with different bodies in the same class, what actually happens in Python? State the rule and its consequence in 1–2 sentences.
- Implement the Calculator class with a single `add` method that supports two or more arguments and satisfies all three behaviors described above. Use an appropriate mechanism in Python to implement method overloading-like behavior based on argument types.
- **Test the method with:**
 - `add(3, 4)`
 - `add(3, 4, 5, 6)`
 - `add("hello", "world")`
 - `add("hello", "world", "python")`
 - `add([1, 2, 3], [2, 3, 4])`
 - `add([1, 2], [2, 3], [4, 5])`
- Test the method with **mismatched types**: `add(3, "4")`
- The program should produce a clear Validation.
- State one limitation of this approach compared with true compile-time method overloading in languages such as Java or C++.

[Method overloading]

5. A **Distance class** is required to represent a distance in meters. Create two or more Distance objects and provide support for performing **arithmetic and comparison operations directly between objects**.

The class should:

- Store the distance value in meters.
- Overload the + operator so that two or more Distance objects can be added.
- Overload the - operator to find the difference between two Distance objects.
- Overload the == operator to check whether two Distance objects represent the same distance.
- Overload the < operator to compare two Distance objects.
- Display the distance in a readable format when the object is printed.

Tasks:

- Create a **Distance** class with a constructor that accepts distance in meters.
 - o Overload the + operator using `__add__()` so that: `d1 + d2` returns a new Distance object containing the sum.
 - o Extend the + operator so that multiple objects can be added: `d1 + d2 + d3`
 - o Overload the - operator using `__sub__()`.
 - o Overload the == operator using `__eq__()`.
 - o Overload the < operator using `__lt__()`.
 - o Overload `__str__()` so that printing an object displays the distance in meters.
- Test the class using at least three objects.
- Explain how Python translates an expression such as `d1 + d2` into a special method call.
- Create a binary file named: **distances.dat**
- Store the Distance objects in the binary file using the pickle module.
- Write the objects.
- Read the objects back.
- Display the distances read from the binary file.
- Store at least three Distance objects and retrieve them from the file.

[operator overloading +File Handling concepts: Binary file, wb, rb, pickle.dump(), pickle.load()]

6. **Problem Statement:** A media player needs to play items from `AudioTrack`, `VideoClip`, and `Podcast` — three classes that share no common ancestor and were written by different teams. A `Playlist` should be able to play any object that exposes a callable `play()` method, and should skip (not crash on) anything that doesn't, printing a message instead.

Tasks:

- Implement **`AudioTrack`**, **`VideoClip`**, and **`Podcast`**, each with its own **`play()`** returning a distinct message, and confirm none of them share a base class other than `Object`.
- Implement **`Playlist.play_all()`** that works correctly across all three types plus a plain string placed in the same list: `not_playable`. Skip the string instead of crashing.
- Print an appropriate message when an object does not provide a callable **`play()`** method.
- Run **`play_all()`** on:

```
[
    AudioTrack(...),
    VideoClip(...),
    Podcast(...),
    "not_playable"
]
```

- Create a CSV file named: **`playlist_history.csv`**
- Store the result of each playlist item in the CSV file.
- Use `csv.writer()` to create the writer.
- Use `writerow()` to write each result.
- Include suitable column headings such as: Type, Status, Message
- Run the playlist again and store all results in the CSV file.
- Read the CSV file.
- Display the stored records.

[duck Typing +File Handling concepts: `csv`, `csv.writer()`, `writerow()`, `reader()`.]